

Spring Framework — Day 22 Notes

Annotations (Day 05): Stereotype Annotations, Component Scan & Annotation-Based Containers

1. What Are Stereotype Annotations?

If we don't want to use an XML configuration file to define our beans, the IoC container still needs some way to know which classes should be treated as beans. This is where stereotype annotations come in — they mark a class as a bean definition, so the IoC container can recognize the class and create an object of it.

There are five stereotype annotations:

- **@Component** — Generic stereotype for any Spring-managed component.
- **@Service** — Indicates a service / business layer component.
- **@Repository** — Indicates a persistence / DAO layer component and provides exception translation.
- **@Controller** — Indicates a Spring MVC controller.
- **@RestController** — A specialized controller that combines **@Controller** and **@ResponseBody** for REST APIs.

Note

All five annotations do the same underlying job — marking the class as a bean definition so the IoC container can detect it. The different names simply communicate the role of the class within the application's layers (service, repository, controller, etc.).

2. The BeanPostProcessor Behind the Scenes

To process certain annotations on beans that already exist, Spring uses a BeanPostProcessor called **CommonAnnotationBeanPostProcessor**.

It mainly handles:

- **@PostConstruct**
- **@PreDestroy**
- **@Resource**

It's important to note that this BeanPostProcessor does **not** search for classes or create beans — it only processes annotations on beans that the container has already created.

3. <context:component-scan>

Instead of defining every bean individually in the XML configuration file, we can use the `<context:component-scan>` tag:

```
<context:component-scan base-package="your.package.name"/>
```

When Spring starts, it scans the specified package and its sub-packages, looking for classes annotated with stereotype annotations such as **@Component**, **@Service**, **@Repository**, and **@Controller**.

Whenever it finds one of these annotations, it registers that class as a bean definition in the Spring IoC container. The IoC container then creates the bean, performs dependency injection, and manages its life cycle.

Where Does @Autowired Fit In?

A natural question arises: `<context:component-scan>` searches for stereotype annotations, but our classes also contain annotations like `@Autowired` and `@Qualifier`. How do these work without `<context:annotation-config>`?

The answer is that `<context:component-scan>` **internally imports** `<context:annotation-config>`, so annotation processing works for every part of the class automatically.

Note

`<context:component-scan>` automatically enables annotation processing (such as `@Autowired`), so we do not need to configure the `BeanPostProcessors` manually.

4. Demo Program — Using XML + @Component

A.java

```
package com.day22;

import org.springframework.stereotype.Component;

@Component
public class A {
    public void m1() {
        System.out.println("A class m1 method");
    }
}
```

application-context.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           https://www.springframework.org/schema/context/spring-context.xsd"
">

    <context:component-scan base-package="com.day22" />

</beans>
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("com/day22/application-co
ntext.xml");

        A bean = context.getBean("a", A.class);
        bean.m1();
    }
}
```

5. Removing XML Entirely — Types of IoC Container

We are still writing `<context:component-scan>` inside an XML file. The next question is: how do we get rid of XML completely? For that, Spring gives us three types of IoC container:

- **ClassPathXmlApplicationContext** — used when you want to work with both annotations and an XML file.
- **FileSystemXmlApplicationContext** — loads the XML configuration from the file system instead of the classpath.
- **AnnotationConfigApplicationContext** — used when you want to work with annotations only, with no XML file at all.

How Does It Work Without XML?

Without XML, we cannot write any BeanPostProcessor configuration. So how do annotations like `@Autowired` and the stereotype annotations still get processed, the way `<context:component-scan>` used to handle it?

The answer is that **AnnotationConfigApplicationContext** enables all the required BeanPostProcessors internally by default. Because of this, no XML configuration file is needed.

Syntax

```
ApplicationContext context =
    new AnnotationConfigApplicationContext("com.day22");
```

Here, we pass our package name so that Spring scans that package for all stereotype annotations present inside the classes.

6. Demo Program — Using AnnotationConfigApplicationContext

A.java

```

package com.day22;

import org.springframework.stereotype.Component;

@Component
public class A {
    public void m1() {
        System.out.println("A class m1 method");
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext("com.day22");

        A bean = context.getBean("a", A.class);
        bean.m1();
    }
}

```

Output

```
A class m1 method
```

7. Changing the Bean Definition Name

By default, the bean name is derived from the class name (starting with a lowercase letter). This can be changed by passing a name into `@Component`, e.g. `@Component("yourChoiceName")`.

A.java

```

package com.day22;

import org.springframework.stereotype.Component;

@Component("myclass")
public class A {
    public void m1() {
        System.out.println("A class m1 method");
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext("com.day22");

        A bean = context.getBean("myclass", A.class);
        bean.m1();
    }
}

```

8. Changing Bean Scope with @Scope

By default, a bean definition's scope is Singleton. Just as we could change the scope to prototype in XML, we can do the same with the @Scope annotation.

A.java

```

package com.day22;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component("myclass")
@Scope("prototype")
public class A {
    public void m1() {
        System.out.println("A class m1 method");
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext("com.day22");

        A bean1 = context.getBean("myclass", A.class);
        A bean2 = context.getBean("myclass", A.class);

        System.out.println(bean1 == bean2);
        System.out.println(bean1.hashCode() + " " + bean2.hashCode());
    }
}

```

Since the scope is prototype, bean1 and bean2 will be different objects. The comparison (bean1 == bean2) prints false, and the two hash codes will differ.

9. Summary — Annotations Covered So Far

- @Required
- @Autowired
- @Qualifier
- @Component
- @Repository
- @Service
- @Controller
- @RestController
- @Scope

We will continue with more annotations in the next session.